New Features of C# 7.0

Introduction

C# 7.0 is planned to release in parts, Microsoft has release Most of its features with *Visual Studio "15" Preview 4*, it was release in last month (Aug 2016), See Release Post

Code simplification and improve performance are the main key role of this build, Tuples, Pattern matching are some of the really good features introduced. Hope you will enjoy these features and be more productive.

Let's see new features of C# 7.0

Using the code

Tuples (with types and literals)

Return multiple values from a method is now a common practice, we generally use custom datatype, out parameters, Dynamic return type or a tuple object but here C# 7.0 brings ***tuple types*** and ***tuple literals*** for you it just return multiple values/ multiple type inform of tuple object. see below snippet

Hide Copy Code

```
( string, string, string, string) getEmpInfo()
{
    //read EmpInfo from database or any other source and just return them
    string strFirstName = "abc";
    string strAddress = "Address";
    string strCity= "City";
    string strState= "State";
     return (strFirstName, strAddress, strCity, strState); // tuple literal
}

//Just call above method and it will return multiple values
 var empInfo= getEmpInfo();
WriteLine("Emp info as  {empInfo .Item1} {empInfo .Item2} {empInfo .Item3} {empInfo
.Item4}.");
```

In above sample we can easily retrieve multiple values from tuples, but Item1, Item2 name are bit ir-relevant so let's assign some meaningful names before return, see below sample

Hide Copy Code

```
(string strFName, string strAdd, string strC, string strSt) getEmpInfo()
{
    //code goes here
}

//Now when you call method get values with specific name as below
var empInfo= getEmpInfo();
WriteLine("Emp info as {empInfo.strFName} {empInfo.strAdd} {empInfo.strC} {empInfo.strSt}.");
```

Additionally you can return their name directly in tuple literal as below

Hide Copy Code

```
return (strFName: strFirstName, strAdd: strAddress, strCity: strC, strState: strSt);
```

Tuples are very useful thing where you can replace hash table or dictionary easily, even you can return multiple values for a single key, Additionally you can use it instead of List where you store multiple values at single position.

.NET also has a Tuple type (See here) but it is a *reference type* and that leads to performance issue, but C# 7.0 bring a Tuple with *value type* which is faster in performance and a mutable type.

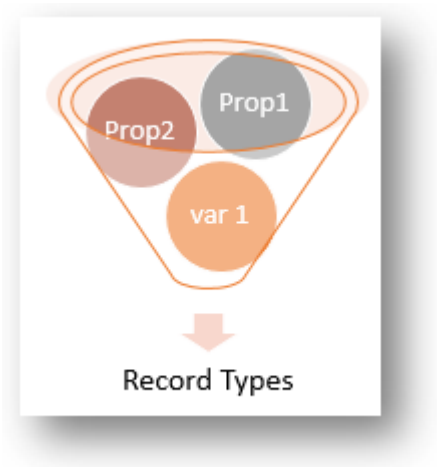
Deconstruction

Most of the time we don't want to access whole tuple bundle or we just need internal values then we can use Deconstruction features of C# 7.0, we can easily de-construct a tuple and fetch values that we need, following snippet will clear your doubt

Hide Copy Code

```
( string strFName,  string strAdd,  string strC, string strSt) = getEmpInfo();
Console.WriteLine($"Address: { strAdd }, Country: { strC }");
```

Record Type



C# support record type, which is nothing but a container of a properties and variables, most of the time classes are full with properties and variables, we need lot of code to just declare them but with the help of *Record Type* you can reduce your effort, see below snippet

Hide Copy Code

```
class studentInfo
{
    string _strFName;
    string _strMName;
    string _strLName;
    studentInfo(string strFN, string strMN, string strLN){
        this._strFName = strFN;
        this._strMName = strMN;
        this._strLName = strLN;
    }
    public string StudentFName {get{ return this._strFName;}}
    public string StudentMName {get{ return this._strMName;}}
    public string StudentLName {get{ return this._strLName;}}
}
```

In above code we have a class with property, constructor and variable, so access and declare variable i need to write more code.

To avoid it i can use *Record Type* in C#, see below snippet

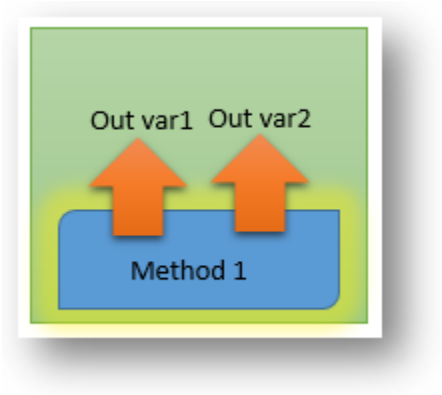
Hide Copy Code

```
class studentInfo(string StudentFName, string StudentMName, string StudentLName);
```

That's it and we have Done !

above snippet produce same output as earlier snippet.

Minimizing OUT



Out parameter is very popular when we want to return multiple values from method, By nature out parameters are ref type and works as an argument, we can use it easily but the only condition is *out* variable should be initialized before it passed. see below snippet

Hide Copy Code

```
class SetOut
{
    void AssignVal(out string strName)
    {
        strName = "I am from OUT";
    }
    static void Main()
    {
        string strArgu;
        AssignVal(out strArgu);
        // here contents of strArgu is "I am from OUT"
    }
}
```

C# 7.0 reduce your pain of writing extra code and you can just pass argument without initialize them, see below snippet

pat

Hide Copy Code

```
  static void Main()
    {
        AssignVal(out string szArgu);
        // here contents of szArgu is "I am from OUT"
```

```
    }
```

You can either use *var* as argument type instead to declare them.

Note that variable are used here, are in limited scope only, thus we can not use them outside method

Since we can define variable as argument directly, C# 7.0 gives us freedom to declare them as *var* also. so you don't need to worry about datatype, see below snippet

Hide Copy Code

```
  static void Main()
    {
        AssignVal(out var szArgu);
        // here contents of szArgu is "I am from OUT"
    }
```

Non-'NULL' able reference type

Null reference is really a headache for all programmers, it is a million dollar exception. If you don't check them you got runtime exception or if you check them for each object then your code goes long and long, To deal with this problem C# 7.0 come with non-nullable reference types

**I think syntax for it yet not fixed still they have release following syntax

'**?**' is for nullable value-type and '**!**' is for non-nullable reference type

Hide Copy Code

```
int objNullVal;     //non-nullable value type
int? objNotNullVal;    //nullable value type
string! objNotNullRef; //non-nullable reference type
string objNullRef;  //nullable reference type
```

Now look at the following complier effect after we run this snippet

Hide Copy Code

```
MyClass objNullRef;  // Nullable reference type
MyClass! objNotNullRef; // Non-nullable reference type

objNullRef = null;   // this is nullable, so no problem in assigning
objNotNullRef = null;   // Error, as objNotNullRef is non-nullable
objNotNullRef = objNullRef;      // Error, as nullable object can not be refered

WriteLine(objNotNullRef.ToString()); // Not null so can convert to tostring
WriteLine(objNullRef.ToString()); // could be null

if (objNullRef != null) { WriteLine(objNullRef.ToString); } // No error as we have already
checked it
WriteLine(objNullRef!.Length); // No error
```

Local Methods/Functions

Local methods and functions is already there in current version of C# (Yes, we can achieve them using Func and Action types, see here Func and Action), but still there are some limitations to local method, we can not have following features in it

- Generic
- out parameters
- Ref
- params

Now with C# 7.0 we can overcome this problems, see below snippet

Hide Copy Code

```
private static void Main(string[] args)
{
    int local_var = 100;
    int LocalFunction(int arg1)
    {
```

```
        return local_var * arg1;
    }

    Console.WriteLine(LocalFunction(100));
}
```

in above snippet we have define 'LocalFunction' as local function which is inside Main Function 'Main', here we can use *out* or *ref* in it.

Readability Improvement with Literals

Many times we use literals in code, if they are too long then we might loose Readability, to sort out such issues C# 7.0 comes with some improvement in Literals. Now C# allows '_' (underscore) in Literals for betterment of understand, it does not effect on its value. see below snippet

Hide Copy Code

```
 var lit1 = 478_1254_3698_44;
 var lit2 = ab0Xab_bc47at_XY;

//C# also come with binary literal for bunary values

 var binLit = 1100_1011_0100_1010_1001;
```

***Literals are nothing but a constant value (hard-coded value) which may be with predefined meaning.* (Litearls in C#)

Pattern matching

C# 7.0 allows user to use pattern in **IS** statement and with **SWITCH** statement, so we can match pattern with any datatype, patterns can be *constant patterns*, *Type patterns, Var patterns.* following sample snippet will clear your concepts, let's start with IS pattern

Hide Copy Code

```
 public  void Method1( object obj)
{
    //following null is constant pattern
     if (obj  is null)  return;
    //datatype pattern, string is datatype that we check directly
     if (obj  is  string st)
    { //code goes here }
    else
    return;
}
```

Switch pattern helps a lot as it uses any datatype for matching additionally 'case' clauses also can have its pattern so it bit flexible implementation

see below snippet

Hide Copy Code

```
class Calculate();
class Add(int a, int b, int c) : Calculate;
class Substract(int a, int b) : Calculate;
class Multiply(int a, int b, int c) : Calculate;

Calculate objCal = new Multiply(2, 3, 4);
switch (objCal)
{
    case Add(int a, int b, int c):
        //code goes here
        break;
    case Substract(int a, int b):
        //code goes here
        break;
    case Multiply(int a, int b, int c):
        //code goes here
        break;
```

```
    default:
        //default case
        break;
}
```

in above sample switch case checks pattern and call 'Multiply' method

'return' by Ref

Have you tried to return your variable from method/function as Ref ? Yes, C# 7.0 allows you to do that. Infect you can pass a variable with **Ref** return them as **Ref** and also store them as **Ref,** isn't it amazing.

see below snippet

Hide Copy Code

```
ref string getFromList(string strVal, string[] Values)
{
 foreach (string val1 in Values)
 {
     if (strVal == val1)
        return ref val1; //return location as ref not actual value
 }
}

string[] values = { "a", "b", "c", "d" };
ref string strSubstitute = ref getFromList("b", values);
strSubstitute = "K"; // replaces 7 with 9 in the array
System.Write(values[1]); // it prints "K"
```

In above sample we have find and replace a string, by return a Ref from method.

Throw Exception from Expression

You read it right, in C# 7.0 Now you can throw exception from your expression directly. see below snippet

Hide Copy Code

```
public string getEmpInfo( string EmpName)
    {
        string[] empArr = EmpName.Split(",");
        return (empArr.Length > 0) ? empArr[0] : throw new Exception("Emp Info Not exist");
    }
```

In above snippet we can directly throw exception from *return* statement, isn't it really good !

Point to be Notice

All above features are expected to be a part of C# 7.0, yet Microsoft has release some of it with *Visual studio 2015 Release 4*.

Hope you enjoy these new features of C# 7.0

- Happy Coding

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)